

GPUMODE Lecture Study Notes: Lecture 45

Outperforming cuBLAS on H100

Core Problem the Lecture Addresses

The lecture studies one narrow but technically deep problem: how to write a highly optimized BF16 matrix multiplication kernel for NVIDIA H100 that can approach or slightly exceed cuBLAS performance for selected matrix sizes.

The target operation is dense GEMM:

$$C = AB,$$

where

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

For the lecture's main setting, the input and output tensors use BF16 storage, while accumulation is performed in FP32:

$$A_{ik}, B_{kj}, C_{ij} \in \text{BF16}, \quad \text{accumulator}_{ij} \in \text{FP32}.$$

The central technical challenge is not the mathematical definition of GEMM. The challenge is mapping GEMM onto H100 hardware so that the tensor cores, Tensor Memory Accelerator, shared memory, registers, L2 cache, and thread block scheduling all work together with minimal stalls.

The lecture shows a progression:

31.8 TFLOP/s $\rightarrow$ 317 TFLOP/s $\rightarrow$ 423 TFLOP/s $\rightarrow$ 631 TFLOP/s $\rightarrow$ 660 TFLOP/s $\rightarrow$ 747 TFLOP/s $\rightarrow$ slight

The important engineering lesson is that modern GPU performance is controlled by several interacting layers:

- tensor core instruction shape,

- register allocation across warp groups,
- shared memory layout,
- asynchronous global-to-shared memory movement,
- producer-consumer pipelining,
- software scheduling across SMs,
- L2 cache locality,
- TMA multicast,
- asynchronous stores,
- low-level PTX instructions unavailable directly through standard CUDA.

Required Background

Naive Matrix Multiplication

The scalar definition of matrix multiplication is

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

A naive CPU-style implementation uses three loops:

```
for (int i = 0; i < M; ++i) {
    for (int j = 0; j < N; ++j) {
        float acc = 0.0f;
        for (int k = 0; k < K; ++k) {
            acc += A[i * K + k] * B[k * N + j];
        }
        C[i * N + j] = acc;
    }
}
```

This implementation is simple but does not expose enough parallelism or data reuse for a GPU. A GPU implementation must divide the output matrix into tiles and assign work to thread blocks, warps, and tensor core instructions.

GPU Execution Model

An NVIDIA GPU consists of many streaming multiprocessors, or SMs. A kernel launch creates a grid of thread blocks. Each thread block is scheduled onto one SM. Threads inside a block execute in groups of 32 called warps.

A simplified hierarchy is:

Grid $\rightarrow$ Thread blocks $\rightarrow$ Warps $\rightarrow$ Threads.

For H100, the lecture uses the mental model of approximately 132 SMs on the SXM variant. Each SM contains:

- registers,
- shared memory,
- warp schedulers,
- tensor cores,
- load-store units,
- access to L1 and L2 cache,
- access to HBM through the memory hierarchy.

Memory Hierarchy

The relevant hierarchy is:

Registers $\rightarrow$ Shared memory $\rightarrow$ L1 cache $\rightarrow$ L2 cache $\rightarrow$ HBM.

Registers are the fastest storage but are private to each thread. Shared memory is explicitly managed by a thread block and is much faster than HBM. L2 is shared across SMs and becomes important when multiple SMs reuse nearby matrix tiles.

Tiling

Instead of computing one scalar C_{ij} at a time, a high-performance GEMM computes a tile:

$$C_{\text{tile}} \in \mathbb{R}^{B_M \times B_N}.$$

To compute this tile, the kernel loops over the reduction dimension K in chunks of size B_K :

$$C_{\text{tile}} = \sum_{t=0}^{K/B_K-1} A_{\text{tile}}^{(t)} B_{\text{tile}}^{(t)}.$$

The tile shapes are:

$$A_{\text{tile}}^{(t)} \in \mathbb{R}^{B_M \times B_K}, \quad B_{\text{tile}}^{(t)} \in \mathbb{R}^{B_K \times B_N}, \quad C_{\text{tile}} \in \mathbb{R}^{B_M \times B_N}.$$

The algorithmic skeleton is:

1. Load a tile of A from HBM to shared memory.
2. Load a tile of B from HBM to shared memory.
3. Multiply the two tiles using tensor cores.
4. Accumulate into registers.
5. Repeat over K .
6. Store the final tile of C back to HBM.

Visual Concept

Draw C as a large matrix divided into rectangular tiles. Highlight one output tile. Then draw the corresponding horizontal tile from A and vertical tile from B that must be multiplied to produce it.

Main Concepts

Why Standard CUDA Is Not Enough

The lecture emphasizes that ordinary CUDA C++ is not the right abstraction level for fully exploiting Hopper. Many important H100 features are exposed through PTX or through libraries such as CUTLASS and CuTe.

The key H100 mechanisms discussed include:

- warp-group matrix multiply-accumulate instructions,
- Tensor Memory Accelerator,
- asynchronous barriers,
- TMA multicast,
- TMA asynchronous stores,
- low-level cache hints.

A recurring theme is that H100 programming often requires manually orchestrating behavior that older GPU programming models hid from the developer.

Tensor Cores and WGMMA

The major performance jump comes from replacing scalar or SIMT-style multiplication with tensor core instructions.

For BF16 on H100, the lecture discusses tensor core instruction shapes with:

$$M_{\text{tc}} = 64, \quad K_{\text{tc}} = 16, \quad N_{\text{tc}} \in \{8, 16, 32, 64, 128, 256\}.$$

A tensor core instruction conceptually computes:

$$D \leftarrow D + A_{\text{frag}} B_{\text{frag}},$$

where

$$A_{\text{frag}} \in \mathbb{R}^{64 \times 16}, \quad B_{\text{frag}} \in \mathbb{R}^{16 \times N_{\text{tc}}}, \quad D \in \mathbb{R}^{64 \times N_{\text{tc}}}.$$

On Hopper, these operations are issued by a warp group rather than a single warp. A warp group contains 128 threads:

$$4 \text{ warps} \times 32 \text{ threads per warp} = 128 \text{ threads}.$$

The accumulator tile is distributed across these 128 threads.

Accumulator Register Pressure

Consider a tensor core output tile:

$$D \in \mathbb{R}^{64 \times 16}.$$

The number of FP32 accumulator elements is:

$$64 \cdot 16 = 1024.$$

If distributed across 128 threads, the average number of accumulator registers per thread is:

$$\frac{1024}{128} = 8.$$

This is very manageable. But for a larger tensor core instruction:

$$D \in \mathbb{R}^{64 \times 256},$$

the number of accumulator elements becomes:

$$64 \cdot 256 = 16384.$$

Distributed over 128 threads, this gives:

$$\frac{16384}{128} = 128$$

FP32 accumulator registers per thread. If the larger output tile is 128×256 , then the accumulator count is:

$$128 \cdot 256 = 32768,$$

and distributed over 128 threads:

$$\frac{32768}{128} = 256.$$

This reaches the approximate practical register limit per thread on H100. In reality, the thread also needs registers for pointers, loop variables, predicates, offsets, and temporary values. Therefore, using a 128×256 tile with a single consumer warp group is not feasible.

Splitting the Output Tile Across Consumers

The lecture resolves this by splitting the 128×256 output tile into two 64×256 tiles:

$$C_{\text{tile}} \in \mathbb{R}^{128 \times 256} \Rightarrow \begin{cases} C_{\text{upper}} \in \mathbb{R}^{64 \times 256}, \\ C_{\text{lower}} \in \mathbb{R}^{64 \times 256}. \end{cases}$$

Each tile is handled by a separate consumer warp group. This allows the kernel to use the largest useful $N_{\text{tc}} = 256$ tensor core instruction without forcing one warp group to hold the entire 128×256 accumulator tile.

This improves tensor core utilization while keeping register pressure feasible.

Visual Concept

Draw one 128×256 output tile. Split it horizontally into two 64×256 rectangles. Label each rectangle as handled by one consumer warp group.

Hardware-Level Explanation

Shared Memory Layout and Swizzling

Tensor cores do not accept arbitrary shared memory layouts. The matrices loaded into shared memory must follow particular layouts that avoid shared memory bank conflicts and match the tensor core instruction requirements.

The lecture points out a practical difficulty: public low-level documentation for some Hopper shared memory layouts has been incorrect or hard to use. The CUTLASS and CuTe libraries contain unit-tested layout logic that avoids many of these pitfalls.

The general problem is that a rectangular tile in global memory:

$$A_{\text{tile}} \in \mathbb{R}^{B_M \times B_K}$$

is not necessarily contiguous in memory. It may consist of many row segments separated by strides. Loading this tile into shared memory requires both:

- globally coalesced reads from HBM,
- a shared memory layout compatible with tensor core access.

This is exactly where TMA becomes important.

Tensor Memory Accelerator

Tensor Memory Accelerator, or TMA, is a Hopper hardware feature for moving multidimensional tensor tiles between global memory and shared memory.

Instead of manually issuing many per-thread loads, the programmer describes a tensor tile and asks TMA to transfer it asynchronously:

HBM tile $\longrightarrow$ shared memory tile.

Conceptually:

```
// Conceptual pseudocode, not exact CUDA syntax.
if (is_producer_thread) {
    tma_load_async(
        shared_memory_destination,
        global_memory_tensor_descriptor,
        tile_coordinates,
        completion_barrier
    );
}
```

The important properties are:

- TMA is asynchronous.
- TMA understands multidimensional tensor layouts.
- TMA can place data into shared memory in layouts suitable for tensor cores.
- TMA can signal completion through barriers.

TMA changes the programming model. The programmer now explicitly manages a pipeline between memory movement and computation.

Producer-Consumer Pipeline

The kernel separates work into producer and consumer warp groups.

- The producer warp group issues TMA loads from HBM to shared memory.
- The consumer warp groups issue WGMMMA tensor core instructions using the loaded shared memory tiles.

The producer and consumers communicate through a shared queue implemented with barriers.

The abstract loop is:

```
// Conceptual GEMM pipeline.
for each output tile assigned to this SM:
    initialize accumulator registers

    for k_tile in range(0, K, BK):
        producer loads A_tile and B_tile into shared memory
        consumer waits until the tile is ready
        consumer performs WGMMMA
        producer advances to the next shared-memory stage

    store accumulator tile to global memory
```

A more realistic pipeline uses multiple shared memory stages:

$$\text{stage count} = S.$$

The producer may fill stage s while the consumer computes from stage $s - 1$. This hides memory latency behind tensor core execution.

$$\text{Useful overlap} = \min(T_{\text{load}}, T_{\text{compute}}).$$

If the load time is mostly hidden, the kernel approaches compute-bound behavior. If tensor cores wait for data, the kernel becomes memory-latency-bound or synchronization-bound.

Why Use Multiples of 128 Threads

The lecture notes that even though a TMA load can be issued by very few threads, Hopper programming guidance often organizes producer and consumer work in units of warp groups, or 128 threads.

This matches WGMMMA execution granularity:

$$1 \text{ warp group} = 128 \text{ threads.}$$

The producer warp group may appear underutilized because the TMA hardware does the real transfer work. However, separating the producer from consumers allows cleaner asynchronous pipelining.

Occupancy Changes on H100 GEMM

Traditional CUDA optimization often emphasizes high occupancy. In this H100 GEMM, occupancy can intentionally be low because each thread block uses:

- many registers,
- large shared memory buffers,
- several warp groups,
- large tile sizes.

The kernel may run approximately one large thread block per SM. That means the programmer must handle scheduling manually instead of relying entirely on the hardware block scheduler.

Mathematical Reasoning

FLOP Count for GEMM

For GEMM:

$$C = AB,$$

each output element performs K multiply-adds. Counting one multiply and one add as two FLOPs:

$$\text{FLOPs} = 2MNK.$$

If the measured runtime is T seconds, throughput is:

$$\text{Throughput} = \frac{2MNK}{T}.$$

In teraflops:

$$\text{TFLOP/s} = \frac{2MNK}{T \cdot 10^{12}}.$$

Arithmetic Intensity

Arithmetic intensity is:

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

For ideal GEMM with each matrix loaded and stored once:

$$\text{Bytes} \approx 2MK + 2KN + 2MN,$$

assuming BF16 storage uses 2 bytes per element. Therefore:

$$\text{AI} \approx \frac{2MNK}{2MK + 2KN + 2MN}.$$

Large GEMMs have high arithmetic intensity and should be compute-bound. But on H100, this simple roofline model is incomplete because power limits, synchronization overhead, shared memory conflicts, register pressure, and instruction scheduling can also limit performance.

Tile-Level Work

For one output tile:

$$C_{\text{tile}} \in \mathbb{R}^{B_M \times B_N},$$

with reduction tile size B_K , each K -stage performs:

$$2B_M B_N B_K$$

FLOPs.

If there are K/B_K stages, then one output tile performs:

$$2B_M B_N K$$

FLOPs.

The shared memory footprint per stage is approximately:

$$\text{SMEM}_{\text{stage}} = 2B_M B_K + 2B_K B_N$$

bytes for BF16 A and B . With S pipeline stages:

$$\text{SMEM}_{\text{pipeline}} = S(2B_M B_K + 2B_K B_N).$$

This is limited by the available shared memory per SM.

Register Footprint

For an FP32 accumulator tile:

$$C_{\text{acc}} \in \mathbb{R}^{B_M \times B_N},$$

the total accumulator register count is:

$$R_{\text{acc,total}} = B_M B_N.$$

Distributed across T participating threads:

$$R_{\text{acc/thread}} = \frac{B_M B_N}{T}.$$

For a warp group:

$$T = 128.$$

Thus:

$$R_{\text{acc/thread}} = \frac{B_M B_N}{128}.$$

This formula is one of the most important constraints in the lecture. It explains why tile shape, tensor core instruction shape, and number of consumer warp groups are tightly coupled.

Performance Relative to cuBLAS

The lecture reports performance as a percentage of cuBLAS:

$$\text{Relative Performance} = \frac{\text{Kernel TFLOP/s}}{\text{cuBLAS TFLOP/s}} \cdot 100\%.$$

For example, if cuBLAS reaches 716 TFLOP/s and a custom kernel reaches 747 TFLOP/s, then:

$$\frac{747}{716} \cdot 100\% \approx 104.3\%.$$

This does not mean the custom kernel is universally better than cuBLAS. It means the custom kernel is better for the tested shapes and assumptions.

Code Walkthrough

Naive Tiled GEMM Skeleton

A conventional tiled GEMM has this high-level form:

```
template<int BM, int BN, int BK>
__global__ void tiled_gemm(
    const __nv_bfloat16* A,
    const __nv_bfloat16* B,
    __nv_bfloat16* C,
    int M, int N, int K
) {
    int block_m = blockIdx.y;
    int block_n = blockIdx.x;

    __shared__ __nv_bfloat16 As[BM][BK];
    __shared__ __nv_bfloat16 Bs[BK][BN];

    float acc[/* per-thread fragment */];

    // Initialize accumulators.
    for (int i = 0; i < FRAG_SIZE; ++i) {
        acc[i] = 0.0f;
    }

    for (int k0 = 0; k0 < K; k0 += BK) {
        // Cooperative global-to-shared loads.
        load_A_tile(As, A, block_m, k0);
        load_B_tile(Bs, B, k0, block_n);

        __syncthreads();

        // Compute using the shared-memory tiles.
        compute_fragment(acc, As, Bs);

        __syncthreads();
    }

    // Store the accumulator fragment to C.
    store_C_fragment(C, acc, block_m, block_n);
}
```

This is the conceptual baseline. The H100 kernel replaces the ordinary cooperative loads with TMA and replaces ordinary fragment computation with WGMMMA.

H100 WGMMMA-Based Skeleton

The lecture's H100-style kernel can be abstracted as:

```
// Conceptual pseudocode.
template<int BM, int BN, int BK, int STAGES>
__global__ void h100_gemm_kernel(
    TensorMap A_desc,
    TensorMap B_desc,
    __nv_bfloat16* C,
    int M, int N, int K
) {
    extern __shared__ unsigned char smem[];

    int warp_group_id = threadIdx.x / 128;

    bool is_producer = (warp_group_id == 0);
    bool is_consumer = !is_producer;

    AccumulatorFragment accum;
    accum.clear();

    for (int tile_id = get_initial_tile(); tile_id < num_tiles; tile_id += gridDim.x) {
        TileCoord coord = schedule(tile_id);

        for (int k0 = 0; k0 < K; k0 += BK) {
            int stage = (k0 / BK) % STAGES;

            if (is_producer) {
                wait_for_empty_stage(stage);
                tma_load_A(stage, A_desc, coord.m, k0);
                tma_load_B(stage, B_desc, k0, coord.n);
                mark_stage_full(stage);
            }

            if (is_consumer) {
                wait_for_full_stage(stage);
                wgmma_mma_async(accum, smem_A(stage), smem_B(stage));
                mark_stage_empty(stage);
            }
        }

        store_output_tile(C, accum, coord);
    }
}
```

```
    }
}
```

Major line-by-line interpretation:

- `warp_group_id` partitions the block into 128-thread warp groups.
- The producer group issues asynchronous TMA loads.
- Consumer groups issue WGMMMA tensor core operations.
- `STAGES` controls the shared memory queue depth.
- The barriers implement ownership transfer between producer and consumers.
- The scheduler maps logical tile IDs to matrix coordinates.
- The final store writes accumulator registers to global memory.

Conceptual WGMMMA Operation

The tensor core operation behaves like:

```
// Conceptual pseudocode.
wgmma_mma_async(
    accum_registers,
    shared_memory_A_tile,
    shared_memory_B_tile,
    accumulate_flag
);
```

Mathematically:

$$D \leftarrow \begin{cases} D + A_{\text{smem}} B_{\text{smem}}, & \text{if accumulation is enabled,} \\ A_{\text{smem}} B_{\text{smem}}, & \text{if accumulation is disabled.} \end{cases}$$

The lecture uses the second form as a micro-optimization for the first K -tile. Instead of explicitly zeroing the accumulator registers, the first WGMMMA instruction overwrites them.

Removing Explicit Accumulator Initialization

A simple implementation does:

```
for (int i = 0; i < NUM_ACCUM; ++i) {
    accum[i] = 0.0f;
}

for (int k0 = 0; k0 < K; k0 += BK) {
```

```
    wmma(accum, A_tile, B_tile, /* accumulate = true */);
}
```

The optimized form does:

```
bool first = true;

for (int k0 = 0; k0 < K; k0 += BK) {
    if (first) {
        wmma(accum, A_tile, B_tile, /* accumulate = false */);
        first = false;
    } else {
        wmma(accum, A_tile, B_tile, /* accumulate = true */);
    }
}
```

This avoids a separate register-clearing step. The performance gain is small, but in an already optimized kernel, small improvements matter.

Skippping L2 Cache on Output Stores

The lecture describes another micro-optimization: when writing C back to global memory, the output is usually not immediately reread by the same GEMM kernel. Therefore, caching the stores in L2 can evict useful A and B data.

The idea is:

```
// Conceptual pseudocode.
store_C_with_cache_hint(
    C_pointer,
    output_value,
    /* cache_global_store_in_L2 = false */
);
```

The goal is to preserve L2 capacity for input tiles:

$$A_{\text{tile}}, B_{\text{tile}} \text{ should be favored over } C_{\text{store}}.$$

Asynchronous Stores Through TMA

A normal store path writes accumulator registers directly to global memory. The optimized path writes registers to shared memory first, then lets TMA asynchronously store the shared memory tile to global memory.

Conceptually:

```
// Step 1: consumers write accumulator registers to shared memory.
store_accumulators_to_smem(output_smem_tile, accum);
```

```
// Step 2: TMA asynchronously stores the tile to global memory.
tma_store_async(C_tensor_map, output_smem_tile, tile_coordinates);
```

```
// Step 3: computation for another tile can overlap with the store.
```

This transforms the final store from a blocking operation into another pipelined operation.

Performance Intuition

Optimization Ladder

The lecture presents a sequence of optimizations. The approximate performance story is:

Optimization	Approximate Result	Main Reason
Naive CUDA-style GEMM	31.8 TFLOP/s	Does not exploit Hopper tensor cores well
Basic WGMMMA	317 TFLOP/s	Uses tensor cores
Tile tuning	423 TFLOP/s	Better tile sizes and reuse
Async loads	70% of cuBLAS	Overlaps TMA loads with WGMMMA
Larger tensor core shapes	88% of cuBLAS	Uses larger N_{tc} shape
Manual scheduling with locality	92% of cuBLAS	Improves L2 reuse
Fast PTX barriers	98% of cuBLAS	Reduces synchronization overhead
TMA multicast	102% of cuBLAS	Shares common loads across SMs
Micro-optimizations	104% of cuBLAS	Removes small overheads
Async stores and Hilbert scheduling	106% or more for selected shapes	Better overlap and L2 locality

Why Tensor Cores Create the First Large Jump

The naive baseline is slow because each thread performs ordinary operations. Tensor cores are specialized matrix multiply units. Replacing scalar multiply-add logic with WGMMMA increases compute throughput dramatically.

The key transition is:

many scalar operations $\Rightarrow$ one large matrix instruction.

Why Async Loading Helps

Without asynchronous loading, the loop resembles:

load $\rightarrow$ compute $\rightarrow$ load $\rightarrow$ compute.

With a producer-consumer pipeline, the loop becomes:

producer: load _{$t+1$} while
consumer: compute _{t} .

The ideal time per stage becomes approximately:

$$T_{\text{stage}} \approx \max(T_{\text{load}}, T_{\text{compute}}),$$

instead of:

$$T_{\text{stage}} \approx T_{\text{load}} + T_{\text{compute}}.$$

Why Larger Tensor Core Shapes Help

Using larger tensor core shapes means each instruction performs more useful work. The shape $m64n256k16$ is powerful because it maximizes the N dimension supported by the BF16 WGMMMA instruction.

However, larger shapes increase accumulator pressure:

$$R_{\text{acc/thread}} = \frac{B_M B_N}{128}.$$

The kernel must split the tile across consumers to avoid exceeding register limits.

Why Manual Scheduling Initially Hurts

When launching exactly one persistent thread block per SM, the programmer takes over scheduling from the CUDA block scheduler. A naive schedule may assign simultaneously running SMs to matrix tiles that are far apart.

If the active tiles are far apart, they load unrelated parts of A and B , reducing L2 cache reuse. This causes the custom scheduler to perform worse than hardware scheduling.

L2-Aware Scheduling

The improved schedule groups simultaneously active tiles into compact rectangles. If multiple SMs operate on neighboring tiles, they reuse nearby rows of A or columns of B .

Suppose two adjacent output tiles share the same A panel:

$$C_1 = A_{\text{panel}}B_1, \quad C_2 = A_{\text{panel}}B_2.$$

Then A_{panel} can remain useful in L2. Similarly, vertical neighbors can share B panels.

Hilbert Curve Scheduling

Hilbert curve scheduling improves spatial locality by visiting nearby tiles consecutively. A simple row-major traversal can make large jumps between rows. A Hilbert traversal tries to preserve locality in two dimensions.

The goal is:

successive tile coordinates should be spatially close.

This increases the chance that recently loaded A or B panels remain in L2.

Visual Concept

Draw a 4×4 tile grid. First draw a row-major path that jumps from the end of one row to the start of the next. Then draw a Hilbert-like path that recursively visits nearby quadrants.

TMA Multicast

TMA multicast allows one TMA load to feed multiple SMs in a thread block cluster.

If two SMs compute two output tiles that share the same B tile:

$$C_0 = A_0B_{\text{shared}}, \quad C_1 = A_1B_{\text{shared}},$$

then both SMs need B_{shared} . Without multicast:

$$\text{SM}_0 \text{ loads } B_{\text{shared}}, \quad \text{SM}_1 \text{ loads } B_{\text{shared}}.$$

With multicast:

one TMA request $\rightarrow$ shared delivery to multiple SMs.

This reduces redundant memory traffic and improves performance.

The lecture reports that a cluster size of two was most useful in practice. Larger clusters may increase communication overhead or reduce the number of usable SMs for the kernel schedule.

Power as a Bottleneck

The lecture also emphasizes that H100 GEMM performance is not only about compute and memory bandwidth. Power can become a limiting factor.

If the GPU reaches its power limit, the effective clock may decrease. Then even if tensor cores are theoretically capable of more throughput, the chip cannot sustain the maximum clock while all units are active.

A simplified relation is:

$$\text{Sustained Throughput} \approx \text{Active Tensor Core Work} \times \text{Sustained Clock}.$$

If power limits reduce the sustained clock, throughput drops.

Common Misconceptions

Misconception: Beating cuBLAS Means the Custom Kernel Is Better

A custom kernel can outperform cuBLAS for one shape and still be less useful in production. cuBLAS is valuable because it is fast across many shapes, layouts, transposition modes, data types, epilogues, and edge cases.

The lecture's kernel is closer to a research and learning artifact:

$$\text{shape-specialized kernel} \neq \text{general production GEMM library}.$$

Misconception: High Occupancy Is Always Required

For simple kernels, high occupancy helps hide latency. For large H100 GEMM kernels, each thread block may use so many registers and so much shared memory that occupancy is intentionally low.

The goal becomes keeping tensor cores continuously active, not maximizing the number of resident thread blocks.

Misconception: Shared Memory Is Automatically Fast

Shared memory can be fast, but only with good layouts. Poor layouts cause bank conflicts. Tensor core instructions also require specific shared memory arrangements. Therefore, shared memory performance depends heavily on layout and access pattern.

Misconception: TMA Is Just a Faster Load

TMA is not merely a faster version of a normal load. It changes the structure of the kernel by enabling asynchronous tensor-tile movement and producer-consumer pipelines.

Misconception: The CUDA Compiler Will Always Find the Best Schedule

For Hopper-specific kernels, the compiler often cannot infer the intended low-level schedule. The programmer may need explicit PTX, barriers, cache hints, and pipeline control.

Misconception: Compute-Bound GEMM Ignores Memory

Even high-arithmetic-intensity GEMM depends on memory hierarchy behavior. L2 reuse, shared memory layout, TMA overlap, and store behavior can all affect tensor core utilization.

Practical Takeaways

- H100 GEMM performance depends on WGMMMA, not ordinary CUDA multiply-adds.
- TMA is essential for efficient global-to-shared tensor tile movement.
- Producer-consumer pipelining hides memory latency behind tensor core work.
- Register pressure determines feasible tile sizes.
- Large tensor core instruction shapes are beneficial, but they require careful splitting across consumer warp groups.
- Manual persistent scheduling can help, but naive scheduling can be much worse than CUDA's default scheduler.
- L2 locality matters even for compute-heavy GEMM.
- TMA multicast is useful when neighboring SMs need the same tile.
- Asynchronous stores help overlap output movement with later computation.

- Micro-optimizations such as skipping explicit accumulator zeroing and avoiding L2 pollution on output stores matter once the main bottlenecks are removed.
- CUTLASS and CuTe are important because they encode correct low-level layout and tiling knowledge.
- A shape-specialized kernel can beat cuBLAS, but cuBLAS remains stronger as a general-purpose production library.

Project or Experiment Ideas

Experiment 1: Naive CUDA GEMM Versus Tiled GEMM

Implement a naive CUDA GEMM and a shared-memory tiled GEMM. Measure:

$$\text{TFLOP/s} = \frac{2MNK}{T \cdot 10^{12}}.$$

Vary B_M , B_N , and B_K . Use Nsight Compute to inspect memory throughput, occupancy, and achieved FLOPs.

Experiment 2: Register Pressure and Tile Size

Write a small CUDA kernel with different accumulator tile sizes per thread. Use compiler output or Nsight Compute to measure registers per thread.

Verify the relationship:

$$R_{\text{acc/thread}} = \frac{B_M B_N}{128}.$$

Experiment 3: L2 Locality Scheduling

Simulate tile traversal orders in Python:

- row-major order,
- column-major order,
- grouped rectangular order,
- Hilbert curve order.

For each traversal, estimate whether consecutive tiles reuse A , B , both, or neither.

Experiment 4: TMA Pipeline Study

Using CUTLASS examples or a minimal Hopper kernel, vary the number of pipeline stages:

$$S \in \{2, 3, 4, 5\}.$$

Measure whether increasing S improves latency hiding or simply consumes too much shared memory.

Experiment 5: Cache Hint Benchmark

Compare output stores that use default caching versus stores that avoid polluting L2. Measure whether preserving L2 for input tiles improves GEMM throughput.

Experiment 6: cuBLAS Versus Shape-Specialized Kernel

Benchmark cuBLAS for fixed square matrix sizes such as:

$$M = N = K \in \{4096, 8192, 16384, 32768\}.$$

Then compare against a specialized kernel or CUTLASS-generated kernel. Observe that cuBLAS is often extremely strong across broad cases, while custom kernels may win on narrow shapes.

Final Mental Model

The lecture's central mental model is:

An H100 GEMM kernel is not just a tiled matrix multiplication. It is a manually scheduled dataflow machine. TMA streams tensor tiles from HBM into shared memory, barriers implement a producer-consumer queue, WGMMMA keeps tensor cores busy, registers hold distributed FP32 accumulators, L2 cache reuse depends on tile scheduling, and multicast reduces redundant loads across neighboring SMs.

At the highest level, the optimized kernel tries to make the following pipeline continuous:

load next tiles || compute current tile || store previous tile.

The kernel becomes fast when tensor cores rarely wait. Every optimization in the lecture serves that goal:

Keep tensor cores fed, keep data movement overlapped, and keep locality high.